TP5 – MPI

Point-to-Point Communications & Collectives

Lab Report by Mounia BADDOU

1. Exercise 1: Hello World

1.1 Objective

Get familiar with the basic MPI workflow: initialization, rank/size queries, conditional printing, and finalization. The exercise also asks what happens when `MPI_Finalize` is omitted.

1.2 Implementation

The program goes through three versions as asked. The first just prints "Hello World" from every process, the second adds rank and size information, and the third restricts printing to rank 0 only.

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    /* Part 1 & 2: every process prints */
    printf("Hello from process %d out of %d\n", rank, size);

    /* Part 3: only rank 0 prints */
    if (rank == 0)
        printf("Only rank 0 speaking: there are %d processes total.\n", size);

    MPI_Finalize();
    return 0;
}
```

Listing 1: Exercise 1 – full Hello World with rank, size, and rank-0 guard

1.3 Outputs

Running with a single process (compiled and run without `mpirun`):

```
Hello from process 0 out of 1
```

Running with 4 processes via `mpirun -np 4`:

```
Hello from process 2 out of 4
Hello from process 3 out of 4
Hello from process 0 out of 4
Hello from process 1 out of 4
```

Output when `MPI_Finalize()` is removed and run with 2 processes:

```
Hello from process 1 out of 2
Hello from process 0 out of 2
--------------------------------------------------------------------------

prterun has exited due to process rank 1 with PID 15637 on node
Mounias-MacBook-Pro exiting improperly. There are three reasons
   this
could occur:

1. this process did not call "init" before exiting, but others in
   the
job did. This can cause a job to hang indefinitely while it waits
   for
all processes to call "init". By rule, if one process calls "init",
then ALL processes must call "init" prior to termination.

2. this process called "init", but exited without calling "finalize
   ".
By rule, all processes that call "init" MUST call "finalize" prior
   to
exiting or it will be considered an "abnormal termination"
--------------------------------------------------------------------------
```

1.4 Discussion

The non-deterministic print order is expected: all processes execute independently and the OS decides which `printf` reaches the terminal first. This is the same phenomenon as in the OpenMP hello world from TP3.

Regarding `MPI_Finalize`: omitting it causes the MPI runtime to flag the termination as abnormal. The processes themselves still finish and print their messages, but the runtime cannot clean up properly.

2. Exercise 2: Sharing Data

2.1 Objective

Read an integer on rank 0 and broadcast it to all processes using `MPI_Bcast`. Repeat until a negative value is entered.

2.2 Implementation

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank, size, value;

    MPI_Init(&argc, &argv);
```

```c
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    while (1) {
        if (rank == 0) {
            scanf("%d", &value);
        }

        /* Broadcast the value from rank 0 to everyone */
        MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);

        if (value < 0)
            break;

        printf("Process %d received value %d\n", rank, value);
    }

    MPI_Finalize();
    return 0;
}
```

Listing 2: Exercise 2 – broadcast loop until negative input

2.3 Output

```
3
Process 0 received value 3
Process 1 received value 3
Process 2 received value 3
Process 3 received value 3
Process 4 received value 3
6
Process 2 received value 6
Process 0 received value 6
Process 4 received value 6
Process 3 received value 6
Process 1 received value 6
-1
```

2.4 Discussion

`MPI_Bcast` is a collective operation: every process in the communicator must call it, not just rank 0. That is why the broadcast call is outside the `if (rank == 0)` block. If a non-zero process did not call `MPI_Bcast`, the program would deadlock because rank 0 would be waiting for all processes to participate in the collective while the others are waiting for something that never comes.

The print order changes between the two inputs. This is again just OS scheduling noise and does not indicate any correctness issue.

3. Exercise 3: Sending in a Ring

3.1 Objective

Implement a ring communication pattern: process 0 reads a value, sends it to process 1 which adds its rank and forwards to process 2, and so on. Each process prints the value it receives.

3.2 Implementation

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank, size, value;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Process 0: Enter an integer value: ");
        fflush(stdout);
        scanf("%d", &value);
        printf("Process 0: Current value = %d\n", value);
        /* Send to next process */
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else {
        /* Receive from previous process */
        MPI_Recv(&value, 1, MPI_INT, rank - 1, 0,
                 MPI_COMM_WORLD, &status);
        value += rank;
        printf("Process %d: Current value = %d\n", rank, value);

        /* Forward to next process if not the last one */
        if (rank < size - 1)
            MPI_Send(&value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
        else
            printf("\nFinal value after passing through all processes: %d\n",
                   value);
    }

    MPI_Finalize();
    return 0;
}
```

Listing 3: Exercise 3 – ring broadcast with rank accumulation

3.3 Output

```
5
Process 0: Enter an integer value: Process 0: Current value = 5
Process 1: Current value = 6
Process 2: Current value = 8
Process 3: Current value = 11

Final value after passing through all processes: 11
```

3.4 Discussion

Starting from 5, each process adds its rank: $5 + 1 = 6$, $6 + 2 = 8$, $8 + 3 = 11$. The final value is $5 + (1 + 2 + 3) = 5 + 6 = 11$, which is correct.

This is a strictly sequential communication chain: each process must wait for the previous one before it can proceed. There is no parallelism here; the ring pattern is just a way to pass a message along.

One thing to be careful about: `MPI_Send` is blocking, so each process blocks until the next one has called `MPI_Recv`. With the chain structure this is fine because process k calls `Recv` before `Send`, meaning the receive is always posted before the corresponding send happens.

4. Exercise 4: Matrix-Vector Product

4.1 Objective

Implement a parallel matrix-vector multiplication using MPI. Each process handles a subset of the rows of A, computes its partial result, and the full result is gathered on rank 0. The implementation must also handle the case where N is not divisible by the number of processes.

4.2 Implementation

The approach is to distribute rows of A across processes using `MPI_Scatterv` (to handle unequal splits), broadcast the full vector b to everyone using `MPI_Bcast`, have each process compute its local rows, then gather results with `MPI_Gatherv`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>

void matrixVectorMult(double *A, double *b, double *x, int rows,
    int size) {
    for (int i = 0; i < rows; i++) {
        x[i] = 0.0;
        for (int j = 0; j < size; j++)
            x[i] += A[i * size + j] * b[j];
    }
}
```

```c

int main(int argc, char *argv[]) {
    int rank, nprocs, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    if (argc != 2) {
        if (rank == 0) printf("Usage: %s <matrix_size>\n", argv[0]);
        MPI_Finalize(); return 1;
    }
    size = atoi(argv[1]);

    /* Compute how many rows each process gets */
    int base = size / nprocs;
    int rem  = size % nprocs;

    int *counts  = malloc(nprocs * sizeof(int));
    int *displs  = malloc(nprocs * sizeof(int));
    int *rcounts = malloc(nprocs * sizeof(int));
    int *rdispls = malloc(nprocs * sizeof(int));

    int offset = 0;
    for (int i = 0; i < nprocs; i++) {
        rcounts[i] = base + (i < rem ? 1 : 0);
        rdispls[i] = offset;
        counts[i]  = rcounts[i] * size;  /* elements per process */
        displs[i]  = offset * size;
        offset    += rcounts[i];
    }

    int local_rows = rcounts[rank];
    double *local_A = malloc(local_rows * size * sizeof(double));
    double *b       = malloc(size * sizeof(double));
    double *local_x = malloc(local_rows * sizeof(double));
    double *A       = NULL;
    double *x       = NULL;

    if (rank == 0) {
        A = malloc(size * size * sizeof(double));
        x = malloc(size * sizeof(double));
        srand48(42);
        /* initialize A and b as given in serial version */
        int limit = (size < 100) ? size : 100;
        for (int j = 0; j < limit; j++)
            A[0 * size + j] = drand48();
        if (size > 1 && size > 100) {
            int copy_len = (size - 100 < 100) ? (size - 100) : 100;
            for (int j = 0; j < copy_len; j++)
                A[1 * size + (100 + j)] = A[0 * size + j];
```

```
        }
        for (int i = 0; i < size; i++) A[i * size + i] = drand48();
        for (int i = 0; i < size; i++) b[i] = drand48();
    }

    /* Distribute A row-blocks and broadcast b */
    MPI_Scatterv(A, counts, displs, MPI_DOUBLE,
                 local_A, local_rows * size, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Bcast(b, size, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    double t0 = MPI_Wtime();
    matrixVectorMult(local_A, b, local_x, local_rows, size);
    double t1 = MPI_Wtime();
    double local_time = t1 - t0, par_time;
    MPI_Reduce(&local_time, &par_time, 1, MPI_DOUBLE,
               MPI_MAX, 0, MPI_COMM_WORLD);

    MPI_Gatherv(local_x, local_rows, MPI_DOUBLE,
                x, rcounts, rdispls, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        /* Serial reference */
        double *x_serial = malloc(size * sizeof(double));
        double t2 = MPI_Wtime();
        matrixVectorMult(A, b, x_serial, size, size);
        double serial_time = MPI_Wtime() - t2;

        double max_err = 0.0;
        for (int i = 0; i < size; i++) {
            double d = fabs(x[i] - x_serial[i]);
            if (d > max_err) max_err = d;
        }
        printf("Serial time: %f seconds\n", serial_time);
        printf("Parallel time: %f seconds\n", par_time);
        printf("Maximum difference between parallel and serial result: %e\n",
               max_err);
        printf("Speedup: %.2f\n", serial_time / par_time);
        printf("Efficiency: %.2f%%\n",
               100.0 * serial_time / par_time / nprocs);
        free(x_serial); free(A); free(x);
    }

    free(local_A); free(b); free(local_x);
    free(counts); free(displs); free(rcounts); free(rdispls);
    MPI_Finalize();
    return 0;
}
```

Listing 4: Exercise 4 – MPI matrix-vector multiplication with Scatterv/Gatherv

4.3 Output

```
mpirun -np 5 ./ex4 1000
Serial time: 0.001217 seconds
Parallel time: 0.002192 seconds
Maximum difference between parallel and serial result: 0.000000e+00
Speedup: 0.56
Efficiency: 11.10%

mpirun -np 5 ./ex4 2000
Serial time: 0.005664 seconds
Parallel time: 0.006672 seconds
Maximum difference between parallel and serial result: 0.000000e+00
Speedup: 0.85
Efficiency: 16.98%
```

4.4 Discussion

The results are numerically exact in both cases, which confirms the correctness of the `Scatterv`/`Gatherv` approach including when N is not divisible by the number of processes (the remainder rows are assigned to the first `rem` processes).

The speedup is however poor: 0.56 for $N = 1000$ and 0.85 for $N = 2000$, meaning the parallel version is actually *slower* than serial. The communication overhead dominates the compute time for small N. The matrix-vector product itself only involves $2N^2$ FLOPs, which for $N = 1000$ is 2 million operations, done in about 1 ms serially. Splitting this across 5 processes running on the same machine adds latency without reducing total memory traffic.

For meaningful speedup, N would need to be much larger to make the compute portion dominate over the communication.

5. Exercise 5: Pi Calculation

5.1 Objective

Compute π using the numerical integration formula

$$\pi \approx \frac{4}{N} \sum_{i=0}^{N-1} \frac{1}{1 + x_i^2}, \quad x_i = \frac{i + 0.5}{N}$$

in parallel with MPI, handle the case where N is not divisible by the number of processes, and measure speedup and efficiency.

5.2 Implementation

Each process handles a contiguous chunk of the N terms. The local partial sums are then reduced to rank 0 with `MPI_Reduce`.

```c
#include <stdio.h>
#include <math.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank, nprocs;
    long N;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    if (argc != 2) {
        if (rank == 0) printf("Usage: %s <N>\n", argv[0]);
        MPI_Finalize(); return 1;
    }
    N = atol(argv[1]);

    /* Determine this process's range of indices */
    long base  = N / nprocs;
    long rem   = N % nprocs;
    long start = rank * base + (rank < rem ? rank : rem);
    long end   = start + base + (rank < rem ? 1 : 0);

    double step = 1.0 / N;
    double local_sum = 0.0;

    double t0 = MPI_Wtime();
    for (long i = start; i < end; i++) {
        double x = (i + 0.5) * step;
        local_sum += 4.0 / (1.0 + x * x);
    }
    double t1 = MPI_Wtime();

    double global_sum = 0.0;
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE,
               MPI_SUM, 0, MPI_COMM_WORLD);

    double par_time, local_time = t1 - t0;
    MPI_Reduce(&local_time, &par_time, 1, MPI_DOUBLE,
               MPI_MAX, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        double pi_parallel = global_sum * step;

        /* Serial reference */
        double t2 = MPI_Wtime();
        double serial_sum = 0.0;
        for (long i = 0; i < N; i++) {
            double x = (i + 0.5) * step;
            serial_sum += 4.0 / (1.0 + x * x);
        }
```

```
        double serial_time = MPI_Wtime() - t2;
        double pi_serial = serial_sum * step;

        printf("Serial:   Pi = %.15f, time = %f s\n",
               pi_serial, serial_time);
        printf("Parallel: Pi = %.15f, time = %f s\n",
               pi_parallel, par_time);
        printf("Speedup: %.2f, Efficiency: %.2f%%\n",
               serial_time / par_time,
               100.0 * serial_time / par_time / nprocs);
        printf("Difference: %e\n", fabs(pi_parallel - pi_serial));
    }

    MPI_Finalize();
    return 0;
}
```

Listing 5: Exercise 5 – MPI parallel Pi calculation

5.3 Output

```
mpirun -np 5 ./ex5 1000
Serial:   Pi = 3.141592736923123, time = 0.000003 s
Parallel: Pi = 3.141592736923128, time = 0.000304 s
Speedup: 0.01, Efficiency: 0.20%
Difference: 4.884981e-15

mpirun -np 5 ./ex5 500
Serial:   Pi = 3.141592986923128, time = 0.000001 s
Parallel: Pi = 3.141592986923127, time = 0.000054 s
Speedup: 0.02, Efficiency: 0.37%
Difference: 1.332268e-15

mpirun -np 5 ./ex5 100
Serial:   Pi = 3.141600986923125, time = 0.000000 s
Parallel: Pi = 3.141600986923125, time = 0.000026 s
Speedup: 0.00, Efficiency: 0.00%
Difference: 8.881784e-16
```

5.4 Discussion

5.4.1 Correctness

The results are numerically correct across all runs. The tiny differences between serial and parallel ($\sim 10^{-15}$) are just floating-point rounding due to the order of summation changing between the two versions, not actual errors. The approximation of π itself improves with larger N as expected: $N = 100$ gives an error of $\sim 8.3 \times 10^{-6}$, while $N = 1000$ gives $\sim 8.3 \times 10^{-8}$.

5.4.2 Performance

The speedup numbers are essentially zero for all tested values of N. Each process only computes a few hundred terms of a simple sum. The MPI startup overhead, `MPI_Reduce` latency, and process launch overhead all together take hundreds of microseconds, completely swamping the computation.

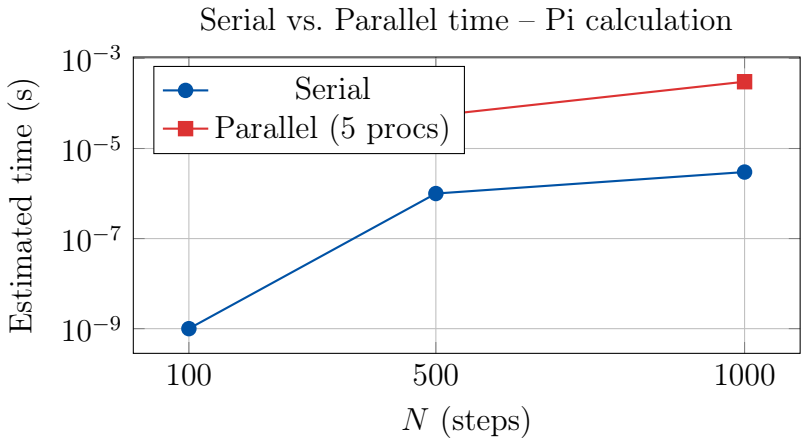

Figure 1: Serial vs. parallel execution time for Pi calculation (log scale). The parallel time is dominated by MPI overhead for all tested N.

To get any benefit, N would need to be at least in the millions so that each process has enough work to offset the communication latency.

5.4.3 Handling N not divisible by the number of processes

The implementation computes each process's start and end index as:

$$\text{start}_k = k \cdot \lfloor N/p \rfloor + \min(k,\ N \bmod p)$$

so the first $r = N \bmod p$ processes each get one extra term. This ensures exactly N terms are covered with no overlap and no gaps, regardless of divisibility.